5W+1H

Memahami framework pengujian unit pada Java

Pengantar. JUnit adalah framework pengujian untuk aplikasi berbasis Java dan JVM. Dengan JUnit, pengembang dapat menulis pemeriksaan otomatis untuk memastikan suatu bagian program menghasilkan perilaku yang sesuai dengan harapan. Pengujian dapat dijalankan berulang kali melalui IDE atau build tool seperti Maven dan Gradle.

1. WHAT — Apa itu JUnit?

JUnit merupakan framework open-source untuk membuat dan menjalankan pengujian otomatis, terutama unit test pada Java. Unit test memeriksa bagian program dalam lingkup kecil—misalnya satu method atau satu kelas—dengan memberikan masukan lalu membandingkan hasil aktual dengan hasil yang diharapkan.

JUnit menyediakan anotasi untuk menandai metode pengujian, assertion untuk memeriksa hasil, serta mekanisme untuk menyiapkan dan membersihkan kondisi pengujian. Pada JUnit modern, API yang umum digunakan adalah JUnit Jupiter, sedangkan JUnit Platform menjadi fondasi untuk menemukan dan menjalankan test.

2. WHO — Siapa yang menggunakan JUnit?

JUnit digunakan oleh programmer Java, software developer, quality assurance (QA) engineer yang melakukan pengujian berbasis kode, serta tim pengembangan yang menerapkan continuous integration/continuous delivery (CI/CD). Pengguna utamanya adalah orang yang menulis atau memelihara kode dan perlu memastikan perubahan tidak merusak fungsi yang sudah ada.

Dalam praktiknya, programmer biasanya menulis unit test bersama kode aplikasi. Reviewer memeriksa kualitas test dalam code review, sementara sistem build atau CI menjalankannya secara otomatis setiap kali ada perubahan kode.

3. WHEN — Kapan JUnit digunakan?

JUnit digunakan sejak pengembangan fitur, setelah perbaikan bug, ketika refactoring, dan setiap kali perubahan kode berisiko memengaruhi perilaku yang sudah berjalan. Unit test idealnya ditulis sedekat mungkin dengan waktu pembuatan kode, bukan hanya menjelang aplikasi selesai.

Contoh: setelah membuat method hitungDiskon(), pengembang langsung menyiapkan test untuk harga normal, diskon nol, batas minimum pembelian, dan nilai yang tidak valid. Test kemudian dijalankan kembali setiap kali method tersebut diubah.

4. WHERE — Di mana JUnit dijalankan?

JUnit dijalankan di lingkungan pengembangan Java yang memiliki JDK dan dependensi JUnit. Pengujian dapat ditulis dan dijalankan melalui IDE seperti IntelliJ IDEA, Eclipse, atau NetBeans; melalui terminal dengan Maven atau Gradle; serta di pipeline CI seperti GitHub Actions.

Dalam proyek Maven, kode aplikasi umumnya disimpan di src/main/java dan kelas pengujian di src/test/java. Pemisahan ini membuat kode produksi dan kode pengujian lebih terorganisasi. JUnit bukan aplikasi untuk menguji tampilan website secara langsung; fokus utamanya adalah memeriksa perilaku kode di sisi program.

5. WHY — Mengapa JUnit penting?

JUnit membantu menemukan kesalahan lebih awal, mengurangi pengujian manual yang berulang, dan memberi keyakinan ketika kode diperbaiki atau dikembangkan. Test yang baik juga menjadi dokumentasi perilaku program karena menunjukkan contoh masukan dan keluaran yang seharusnya.

JUnit dapat dijalankan berkali-kali dengan cara yang konsisten dan diintegrasikan ke proses build. Namun, JUnit tidak menjamin aplikasi bebas bug. Kualitas hasil sangat bergantung pada cakupan dan ketepatan skenario test; unit test juga tidak menggantikan pengujian integrasi, sistem, keamanan, atau usability.

6. HOW — Bagaimana cara kerja JUnit?

Secara umum, pengembang menulis metode test, memberi anotasi @Test, menjalankannya melalui IDE atau build tool, lalu JUnit mengeksekusi metode tersebut dan melaporkan hasilnya. Assertion seperti assertEquals() membandingkan hasil aktual dengan nilai yang diharapkan. Jika cocok, test lulus (passed); jika tidak, test gagal (failed).

Contoh singkat

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class Kalkulator {
    int tambah(int a, int b) { return a + b; }
}

class KalkulatorTest {
    @Test
    void penjumlahanDuaBilangan() {
        Kalkulator kalkulator = new Kalkulator();
        assertEquals(15, kalkulator.tambah(7, 8));
    }
}
```

Penjelasan: Kelas Kalkulator berisi method tambah() yang menjadi objek pengujian. @Test menandai metode test, sedangkan assertEquals(15, kalkulator.tambah(7, 8)) memeriksa apakah hasil pemanggilan method sama dengan nilai yang diharapkan. Jika sama, test passed; jika berbeda, JUnit melaporkan failed.

Anotasi dan assertion yang sering digunakan

Anotasi	Kegunaan
@Test	Menandai sebuah metode sebagai test.
@BeforeEach	Dijalankan sebelum setiap metode test.
@AfterEach	Dijalankan setelah setiap metode test.
@BeforeAll / @AfterAll	Dijalankan sekali sebelum / setelah seluruh test dalam kelas.
assertEquals / assertTrue	Memeriksa kesamaan nilai / kondisi benar.
assertThrows	Memastikan suatu operasi menghasilkan exception yang diharapkan.

Kesimpulan

JUnit memudahkan pengujian otomatis terhadap bagian kecil program Java. Dengan menulis skenario yang jelas, menggunakan assertion yang tepat, dan menjalankan test secara rutin, pengembang dapat lebih cepat mengetahui kesalahan serta menjaga agar perubahan kode tidak merusak fungsi yang telah berjalan.

Referensi

JUnit Team. JUnit — The Programmer-Friendly Testing Framework for Java and the JVM. https://junit.org/

JUnit Team. JUnit User Guide. https://docs.junit.org/current/user-guide/

JetBrains. Tutorial: Get started with JUnit. https://www.jetbrains.com/help/idea/junit.html